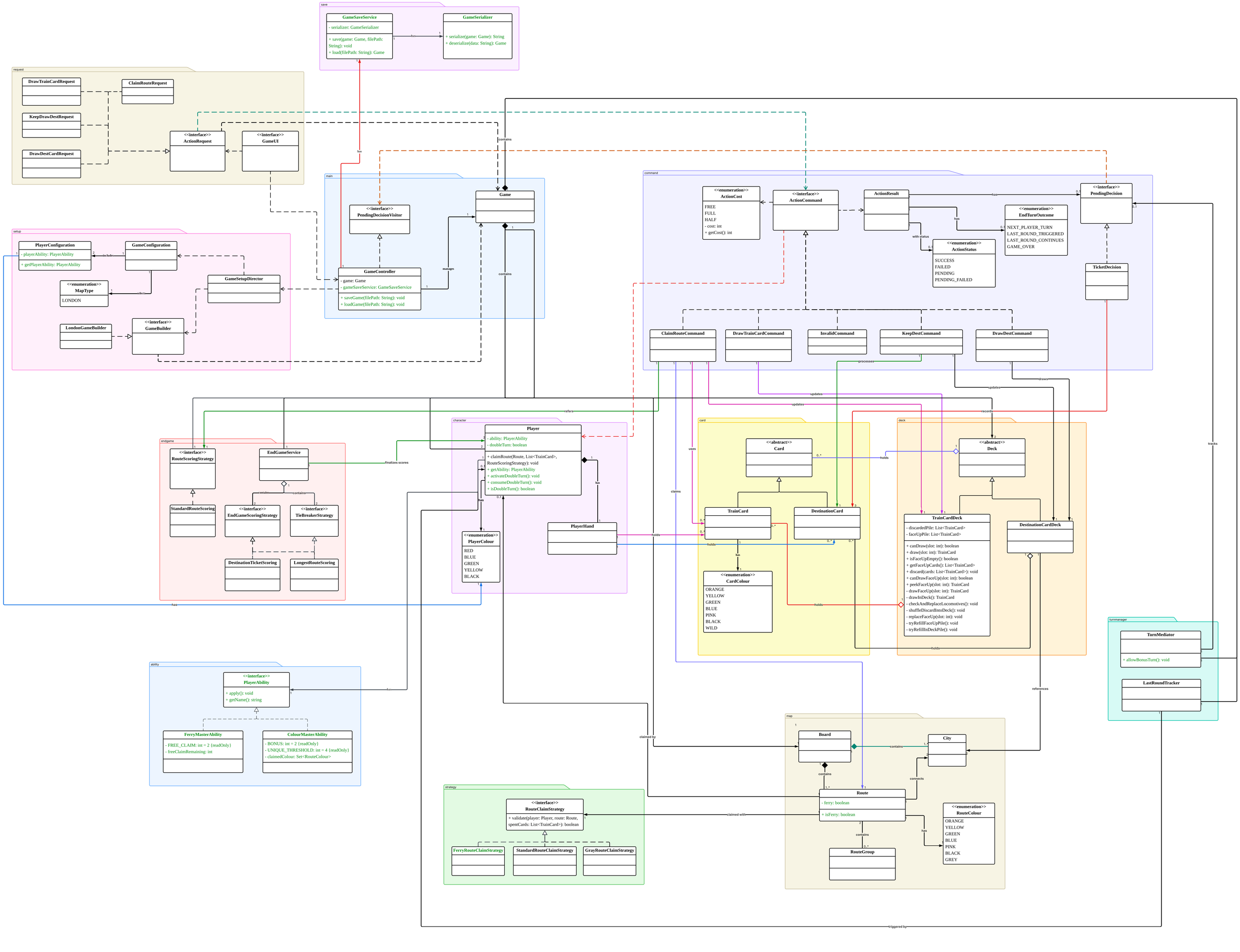

save
GameSaveService
- serializer: GameSerializer
+ save(game: Game, filePath: String): void
+ load(filePath: String): Game
GameSerializer
+ serialize(game: Game): String
+ deserialize(data: String): Game
has
request
DrawTrainCardRequest
ClaimRouteRequest
KeepDrawDestRequest
DrawDestCardRequest
<<interface>> ActionRequest
<<interface>> GameUI
contains
has
main
Game
<<interface>> PendingDecisionVisitor
GameController
- game: Game
- gameSaveService: GameSaveService
+ saveGame(filePath: String): void
+ loadGame(filePath: String): void
manages
contains
setup
PlayerConfiguration
- playerAbility: PlayerAbility
+ getPlayerAbility: PlayerAbility
GameConfiguration
includes
<<enumeration>> MapType
LONDON
selects
GameSetupDirector
LondonGameBuilder
<<interface>> GameBuilder
command
<<enumeration>> ActionCost
FREE
FULL
HALF
- cost: int
+ getCost(): int
<<interface>> ActionCommand
ActionResult
has
<<interface>> PendingDecision
<<enumeration>> EndTurnOutcome
NEXT_PLAYER_TURN
LAST_ROUND_TRIGGERED
LAST_ROUND_CONTINUES
GAME_OVER
with status
<<enumeration>> ActionStatus
SUCCESS
FAILED
PENDING
PENDING_FAILED
TicketDecision
ClaimRouteCommand
DrawTrainCardCommand
InvalidCommand
KeepDestCommand
DrawDestCommand
processes
updates
draws
refers
records
tracks
endgame
<<interface>> RouteScoringStrategy
EndGameService
finalizes scores
contains
StandardRouteScoring
<<interface>> EndGameScoringStrategy
<<interface>> TieBreakerStrategy
DestinationTicketScoring
LongestRouteScoring
character
Player
- ability: PlayerAbility
- doubleTurn: boolean
+ claimRoute(Route, List<TrainCard>, RouteScoringStrategy): void
+ getAbility: PlayerAbility
+ activateDoubleTurn(): void
+ consumeDoubleTurn(): void
+ isDoubleTurn(): boolean
has
<<enumeration>> PlayerColour
RED
BLUE
GREEN
YELLOW
BLACK
PlayerHand
holds
card
<<abstract>> Card
uses
claims
TrainCard
DestinationCard
<<enumeration>> CardColour
ORANGE
YELLOW
GREEN
BLUE
PINK
BLACK
WILD
deck
<<abstract>> Deck
holds
TrainCardDeck
- discardedPile: List<TrainCard>
- faceUpPile: List<TrainCard>
+ canDraw(slot: int): boolean
+ draw(slot: int): TrainCard
+ isFaceUpEmpty(): boolean
+ getFaceUpCards(): List<TrainCard>
+ discard(cards: List<TrainCard>): void
+ canDrawFaceUp(slot: int): boolean
+ peekFaceUp(slot: int): TrainCard
- drawFaceUp(slot: int): TrainCard
- drawInDeck(): TrainCard
- checkAndReplaceLocomotives(): void
- shuffleDiscardIntoDeck(): void
- replaceFaceUp(slot: int): void
- tryRefillFaceUpPile(): void
- tryRefillInDeckPile(): void
DestinationCardDeck
turnmanager
TurnMediator
+ allowBonusTurn(): void
LastRoundTracker
triggers
references
ability
<<interface>> PlayerAbility
+ apply(): void
+ getName(): string
has
FerryMasterAbility
- FREE_CLAIM: int = 2 {readOnly}
- freeClaimRemaining: int
ColourMasterAbility
- BONUS: int = 2 {readOnly}
- UNIQUE_THRESHOLD: int = 4 {readOnly}
- claimedColour: Set<RouteColour>
map
Board
contains
City
connects
Route
- ferry: boolean
+ isFerry: boolean
claimed by
claimed with
RouteGroup
has
<<enumeration>> RouteColour
ORANGE
YELLOW
GREEN
BLUE
PINK
BLACK
GREY
strategy
<<interface>> RouteClaimStrategy
+ validate(player: Player, route: Route, spentCards: List<TrainCard>): boolean
FerryRouteClaimStrategy
StandardRouteClaimStrategy
GrayRouteClaimStrategy
triggered by